dati-basati

A Typst package to draw ER diagrams.

https://github.com/victuarvi/dati-basati

Version 0.1.1

Manual

1. Import 2
2. Basic usage 3
   2.1. Entities 3
      2.1.1. Entity with attributes . . . 3
      2.1.2. Subentities 5
      2.1.3. Attributes positioning . . . 7
   2.2. Relations 10
3. Formatting & customization 13
4. Advanced usage 16
   4.1. Weak entities 16
   4.2. Unorthodox relation position . 18
   4.3. Exceptions 18

Documentation

5. User functions 20
   5.1 dati-basati 20
   5.2 er-diagram 21
   5.3 entity 21
   5.4 subentities 23
   5.5 relation 24

1. Import

Depending on your needs there are different types of imports:

- Import *all* the functions (recommended[1]):

```typ
#import "@preview/dati-basati:0.1.1": *
```

- Import *only* the functions you want to use:

```typ
#import "@preview/dati-basati:0.1.1": er-diagram, entity, relation
```

- Import and (optionally) rename:

```typ
#import "@preview/dati-basati:0.1.1" as renamed
```

  but your code would be more verbose, like in the following example:

```typ
#renamed.er-diagram({
  renamed.entity(
    (0, 0),
    name: "entity0",
  )
})
```

The package is designed to be as straightforward and intuitive as possible, so you should be able to just write diagrams without even needing to read the manual – or at least that was the design goal.

[1] Since it's unlikely there will be conflicts with identically named functions.

2. Basic usage

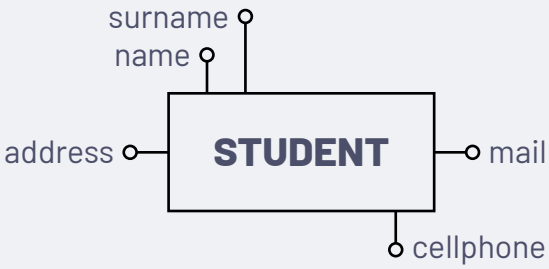
2.1. Entities

A basic entity would appear like this:

```
#er-diagram({
  entity(
    // coordinates
    (0, 0),
    // the internal reference
    name: "entity",
    // the content label
    label: "Nice Name"
  )
})
```

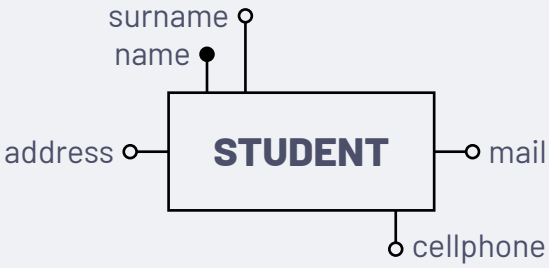
2.1.1. Entity with attributes

```
#er-diagram({
  entity(
    (0, 0),
    name: "student",
    label: "student",
    attributes: (
      north: ("name", "surname"),
      east: ("mail",),
      west: ("address",),
      south: ("cellphone",),
    ),
  )
})
```



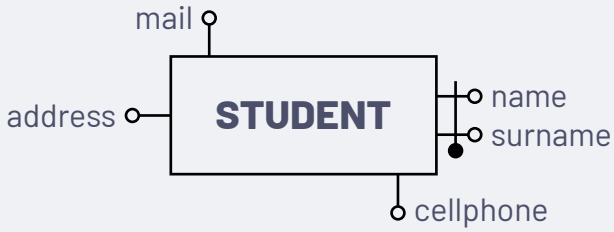
Primary key Adding a single primary key:

```
#er-diagram({
  entity(
    (0, 0),
    name: "student",
    label: "student",
    attributes: (
      north: ("name", "surname"),
      east: ("mail",),
      west: ("address",),
      south: ("cellphone",),
    ),
    primary-key: "name"
  )
})
```



Composite primary key Adding a composite primary key:

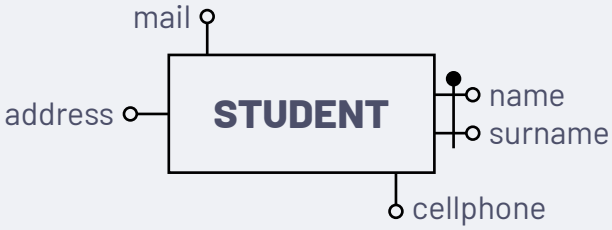
```
#er-diagram({
  entity(
    (0, 0),
    name: "student",
    label: "student",
    attributes: (
      north: ("mail",),
      east: ("name", "surname"),
      west: ("address",),
      south: ("cellphone",),
    ),
    primary-key: ("name", "surname")
  )
})
```



Info

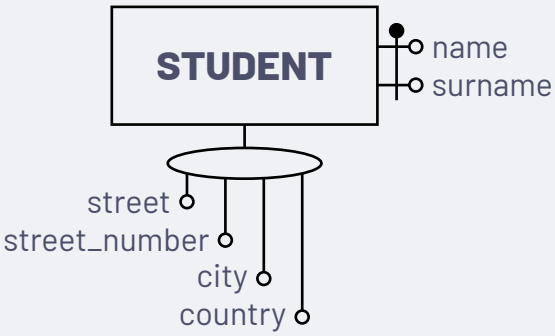
The direction of the line will depend on the exact order you give the attributes:

```
#er-diagram({
  entity(
    (0, 0),
    name: "student",
    label: "student",
    attributes: (
      north: ("mail",),
      east: ("name", "surname"),
      west: ("address",),
      south: ("cellphone",),
    ),
    primary-key: ("name", "surname").rev()
  )
})
```



Composite attributes You might want to group some attributes together:

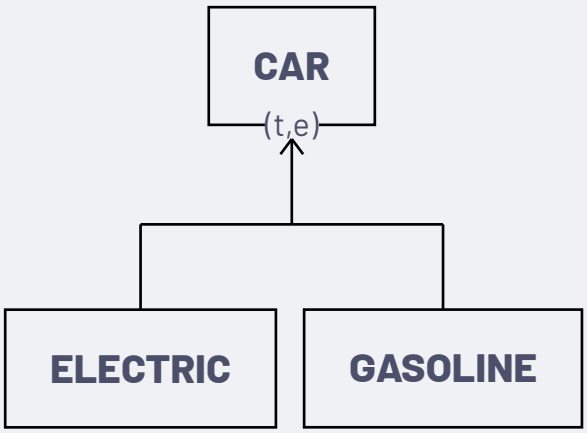
```
#er-diagram({
  entity(
    (0, 0),
    name: "student",
    label: "student",
    attributes: (
      east: ("name", "surname"),
      south: (
        // instead of a string, the comp.
        // attr. is an array
        ("street", "street_number",
        "city", "country"),
      ),
    ),
    primary-key: ("name", "surname").rev()
  )
})
```



> **Info**
>
> The composite attributes is designed to be *the only attribute* on its side, meaning that other attributes on the same side will NOT be drawn.

2.1.2. Subentities

```
#er-diagram({
  entity(
    (0, 4),
    name: "A",
    label: "car",
  )
  entity(
    (-2, 0),
    name: "B",
    label: "electric",
  )
  entity(
    (2, 0),
    name: "C",
    label: "gasoline",
  )
  subentities(
    entity: "A",
    subentities: ("B", "C"),
  )
})
```

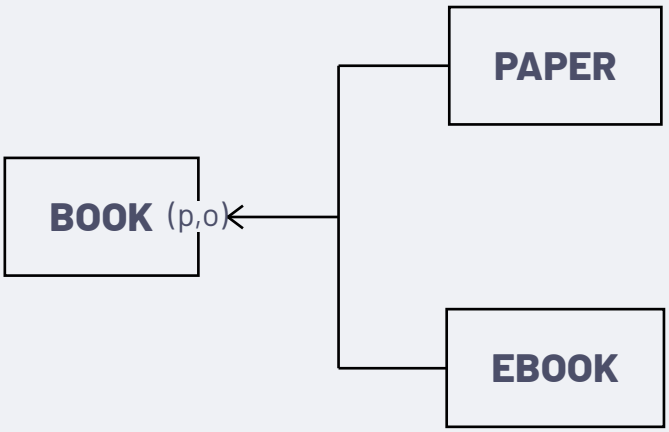


> ⚠ **Warning**
>
> **You can't** connect a subentity to an entity if that entity has not been drawn yet:

```typ
// this will NOT work
#er-diagram({
  subentities(
    entity: "A",
    subentities: ("B", "C"),
  )
  entity(
    (0, 0),
    name: "A",
    label: "entity1",
  )
  entity(
    (-2, 4),
    name: "B",
    label: "entity2",
  )
})
```

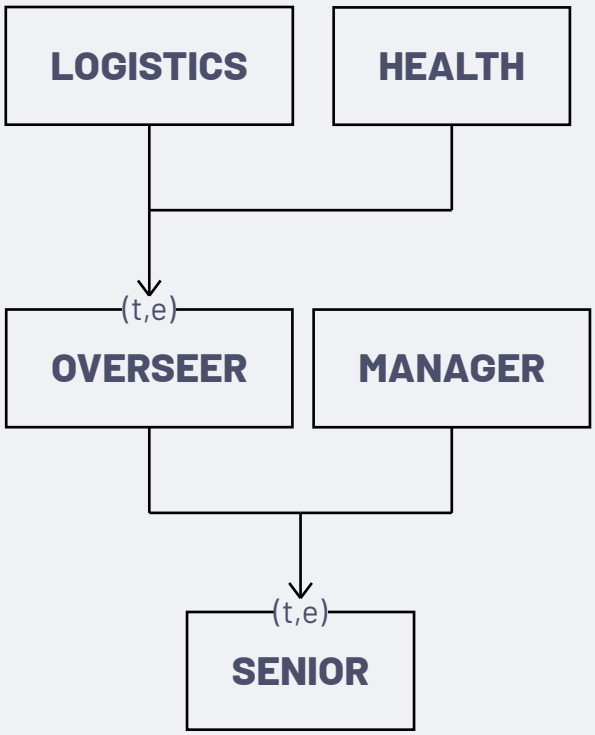
Although subentities generally tend to be below their superentity, you can place them wherever you like best:

```typ
#er-diagram({
  entity(
    (0, 0),
    name: "A",
    label: "book",
  )
  entity(
    (6, 2),
    name: "B",
    label: "paper",
  )
  entity(
    (6, -2),
    name: "C",
    label: "ebook",
  )
  subentities(
    hierarchy: "(p,o)",
    entity: "A",
    subentities: ("B", "C"),
  )
})
```



A more complex layout:

```
#er-diagram({
  entity(
    (0, 0),
    name: "A",
    label: "senior",
  )
  entity(
    (-2, 4),
    name: "B",
    label: "overseer",
  )
  entity(
    (2, 4),
    name: "C",
    label: "manager",
  )
  entity(
    (2, 8),
    name: "D",
    label: "health",
  )
  entity(
    (-2, 8),
    name: "E",
    label: "logistics",
  )
  subentities(
    entity: "A",
    subentities: ("B", "C"),
  )
  subentities(
    entity: "B",
    subentities: ("D", "E"),
  )
})
```



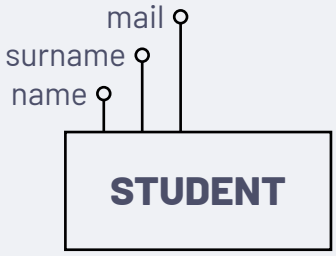
2.1.3. Attributes positioning

Depending on what you are trying to achieve, the default placement could be wrong – you can modify if with `attributes-position`:

```
attributes-position: (
  side: (
    alignment: left | center | right,
    dir: "ltr" | "rtl",                // left-to-right, right-to-left
    start: "from-short" | "from-long" // from shortest to longest
  ),
)
```

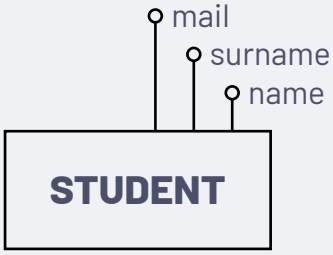
Default:

```
#er-diagram({
  entity(
    (0, 0),
    name: "student",
    label: "student",
    attributes: (
      north: ("name", "surname", "mail"),
    ),
  )
})
```



Mirrored:

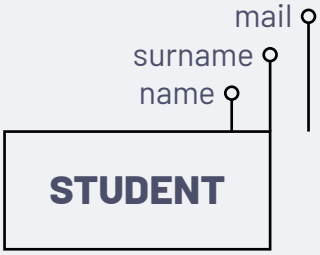
```
#er-diagram({
  entity(
    (0, 0),
    name: "student",
    label: "student",
    attributes: (
      north: ("name", "surname", "mail"),
    ),
    attributes-position: (
      north: (alignment: right, dir: "rtl")
    ),
  )
})
```



Warning

You will find out *some combinations* will lead to visual errors:

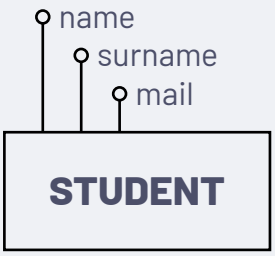
```
#er-diagram({
  entity(
    (0, 0),
    name: "student",
    label: "student",
    attributes: (
      north: ("name", "surname", "mail"),
    ),
    attributes-position: (
      north: (alignment: right)
    )
  )
})
```



In the future this might be patched.

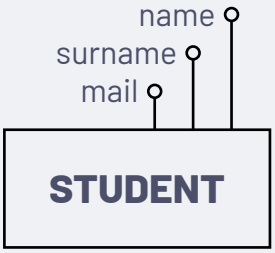
From high to low:

```
#er-diagram({
  entity(
    (0, 0),
    name: "student",
    label: "student",
    attributes: (
      north: ("name", "surname", "mail"),
    ),
    attributes-position: (
      north: (start: "from-long")
    )
  )
})
```



Mirrored:

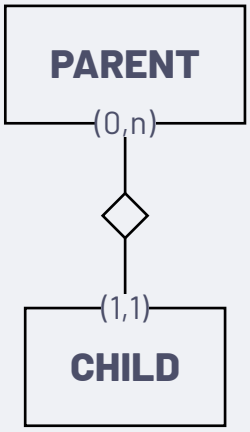
```
#er-diagram({
  entity(
    (0, 0),
    name: "student",
    label: "student",
    attributes: (
      north: ("name", "surname", "mail"),
    ),
    attributes-position: (
      north: (
        start: "from-long",
        alignment: right,
        dir: "rtl"
      )
    )
  )
})
```



2.2. Relations

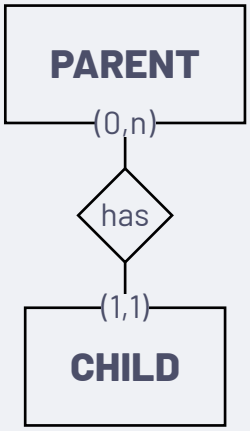
Ideally the relation should be a straight line connecting two entities:

```
#er-diagram({
  entity(
    (0, 0),
    name: "child",
    label: "child",
  )
  entity(
    (0, 4),
    name: "parent",
    label: "parent",
  )
  relation(
    entities: ("parent", "child"),
    cardinality: ("(0,n)", "(1,1)"),
  )
})
```



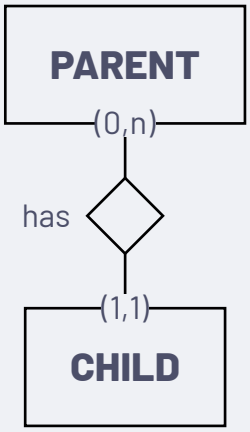
With label:

```
#er-diagram({
  entity(
    (0, 0),
    name: "child",
    label: "child",
  )
  entity(
    (0, 4),
    name: "parent",
    label: "parent",
  )
  relation(
    label: "has",
    entities: ("parent", "child"),
    cardinality: ("(0,n)", "(1,1)"),
  )
})
```



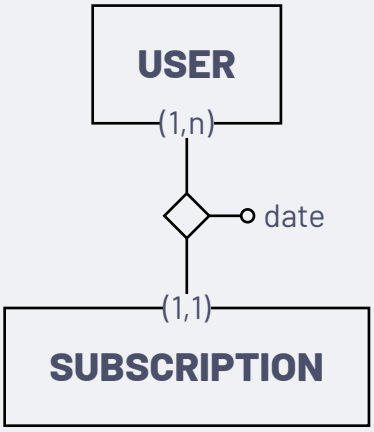
With label outside:

```
#er-diagram({
  entity(
    (0, 0),
    name: "child",
    label: "child",
  )
  entity(
    (0, 4),
    name: "parent",
    label: "parent",
  )
  relation(
    label: ("has", "east"),
    entities: ("parent", "child"),
    cardinality: ("(0,n)", "(1,1)"),
  )
})
```



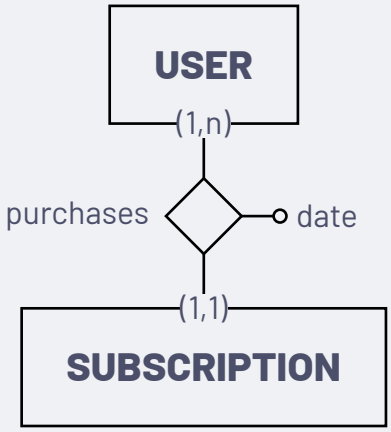
With an attribute (same API as entities, see Section 2.1.1):

```
#er-diagram({
  entity(
    (0, 0),
    name: "user",
    label: "user",
  )
  entity(
    (0, -4),
    name: "subscription",
    label: "subscription",
  )
  relation(
    entities: ("user", "subscription"),
    cardinality: ("(1,n)", "(1,1)"),
    attributes: (east: ("date",)),
  )
})
```



Combined:

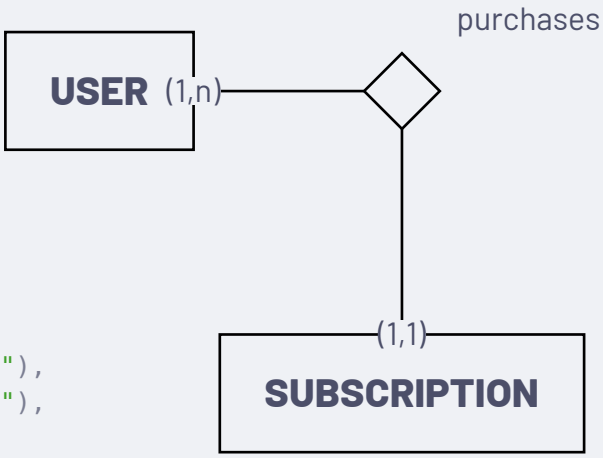
```
#er-diagram({
  entity(
    (0, 0),
    name: "user",
    label: "user",
  )
  entity(
    (0, -4),
    name: "subscription",
    label: "subscription",
  )
  relation(
    label: ("purchases", "east"),
    entities: ("user", "subscription"),
    cardinality: ("(1,n)", "(1,1)"),
    attributes: (east: ("date",)),
  )
})
```



> **Info**
>
> Relations **can't** have primary keys; also there is no such thing as a "weak relation" (see Section 4.1).

Manual placement[2] If you need to manually specify where to place a relation you can:

```
#er-diagram({
  entity(
    (0, 0),
    name: "user",
    label: "user",
  )
  entity(
    (4, -4),
    name: "subscription",
    label: "subscription",
  )
  relation(
    coordinates: (4,0),
    label: ("purchases", "south-west"),
    entities: ("user", "subscription"),
    cardinality: ("(1,n)", "(1,1)"),
  )
})
```



[2] Since the relation attributes API is the same as entities, Section 2.1.3 would apply as expected. Though remember the relations don't have... many anchors to draw from.

3. Formatting & customization

The package provides an handful of themes you can use by importing the corresponding module and applying the show rule:

typ

```typ
#import "@preview/dati-basati:0.1.1" as db
#show: db.with(..db.themes.at("theme-name"))
```

Currently the themes are:

- tiw
- snow
- ghibli
- tiramisu
- futurama
- C62-50
- C62-48
- polimi

The theme schema is as follows:

typ

```typ
#let default-settings = (
  fill: (
    entities: none,
    relations: auto, // auto inherits entities
    primary-key: black,
    weak-entity: auto, // auto inherits primary-key
    cardinality: auto, // auto = page fill
    hierarchy: auto, // auto inherits cardinality
    composite-attributes: auto, // auto inherits cardinality
  ),
  stroke: (
    entities: black,
    relations: black, // auto inherits entities
    lines: auto, // auto inherits entities
    attributes: black,
    composite-attributes: auto, // auto inherits attributes
    primary-key: auto, // auto inherits attributes
    weak-entity: auto, // auto inherits primary-key
    cardinality: none,
    hierarchy: auto, // auto inherits cardinality
  ),
  radius: (
    entities: 0pt,
    cardinality: 0pt,
    hierarchy: auto, // auto inherits cardinality
  ),
  spacing: (
    in-between: (x: 1.2em, y: 1.2em), // spacing between attributes
    padding: 1.2em, // distance from the entity
  ),
  text: (
    entities: l => {
      set par(leading: 0.35em)
      text(
```

```
        top-edge: "bounds",
        size: 1.5em,
        weight: "bold",
        smallcaps(lower(l)),
      )
    },
    relations: l => l,
    attributes: l => l,
    cardinality: l => text(top-edge: "bounds", bottom-edge: "bounds", l),
    hierarchy: auto,
  ),
  attributes-position: (
    "north": (
      alignment: left,
      dir: "ltr",
      start: "from-short",
    ),
    "east": (
      alignment: center,
      dir: "ltr",
    ),
    "south": (
      alignment: right,
      dir: "rtl",
      start: "from-short",
    ),
    "west": (
      alignment: center,
      dir: "rtl",
    ),
  ),
  misc: (
    weak-entities-stroke: false, // double stroke weak entities
    relations-intersection: "|-",
  ),
)
```

You can override single settings:

typ

```
#import "@preview/dati-basati:0.1.1" as db
#show: db.with(..db.themes.at("theme-name"),entities: (fill: green))
```

Or create new themes following the schema:

typ

```
#import "@preview/dati-basati:0.1.1" as db
#import "@preview/catppuccin:1.0.1": *
#let palette = flavors.latte.colors
#show: db.with(
  fill: (
    cardinality: palette.flamingo.rgb,
```

```
    entities: palette.red.rgb,
    relations: palette.red.rgb,
    attributes: palette.red.rgb,
    primary-key: palette.maroon.rgb,
    weak-entity: palette.red.rgb,
  ),
  stroke: (
    cardinality: palette.surface0.rgb + 0.1pt,
    entities: black,
    relations: black,
    attributes: palette.rosewater.rgb,
    weak-entity: orange,
  ),
  text: (
    entities: l => text(
      fill: palette.surface0.rgb,
      weight: 500,
      size: 1.2em,
      smallcaps(l),
    ),
    cardinality: l => text(
      top-edge: "bounds",
      bottom-edge: "bounds",
      fill: palette.surface0.rgb,
      l,
    ),
  ),
  radius: (
    entities: 6pt,
    cardinality: 6pt,
  ),
)
```

There are plenty of examples with each and every theme – and corresponding source code – in `/examples`.

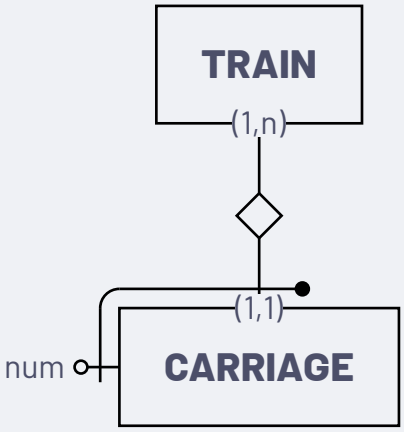
> 🔥 **Tip**
>
> If you need different ER diagrams to have different themes, create a `.typ` file and then include that – in this way, you can apply the `show` rule with the specific theme in the scope of the single file.
>
> In the future, the theming settings could be implemented per `er-diagram`.

4. Advanced usage

4.1. Weak entities

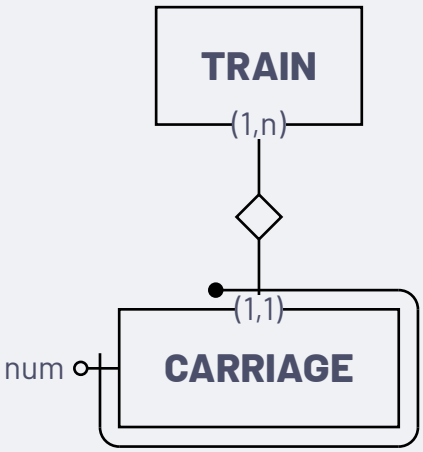
Sometimes there is the need to express a weak costraint, such a carriage number *within the context* of a train; in order to do so specify the weak entity argument with the `(attribute, strong-entity, direction)` format:

```
#er-diagram({
  entity(
    (0, 0),
    name: "train",
    label: "train",
  )
  entity(
    (0, -4),
    name: "carriage",
    label: "carriage",
    attributes: (west: ("num", )),
    weak-entity: ("num", "train")
  )
  relation(
    entities: ("train", "carriage"),
    cardinality: ("(1,n)", "(1,1)"),
  )
})
```



Changing the direction – now CCW, counter clockwise:

```
#er-diagram({
  entity(
    (0, 0),
    name: "train",
    label: "train",
  )
  entity(
    (0, -4),
    name: "carriage",
    label: "carriage",
    attributes: (west: ("num", )),
    weak-entity: ("num", "train", "CCW")
  )
  relation(
    entities: ("train", "carriage"),
    cardinality: ("(1,n)", "(1,1)"),
  )
})
```



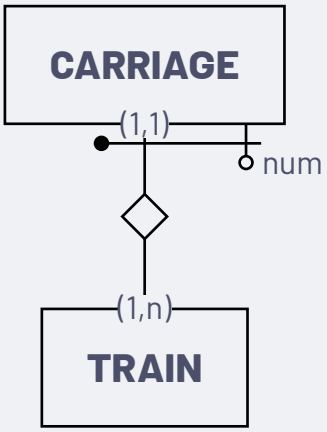
Warning

By default, you can't relate a weak entity to a strong entity *if the strong entity does not exist yet* – this behaviour mirrors how a possible SQL translation would work:

```sql
create table train(
  id int primary key
);
create table carriage(
  id int primary key,
  foreign key(id) references train(id)
)
```

If you need to, you can specify the location of the strong entity via cardinal points:

```
#er-diagram({
  entity(
    (0, 0),
    name: "carriage",
    label: "carriage",
    attributes: (south: ("num", )),
    weak-entity: ("num", "south")
  )
  entity(
    (0, -4),
    name: "train",
    label: "train",
  )
  relation(
    entities: ("train", "carriage"),
    cardinality: ("(1,n)", "(1,1)"),
  )
})
```

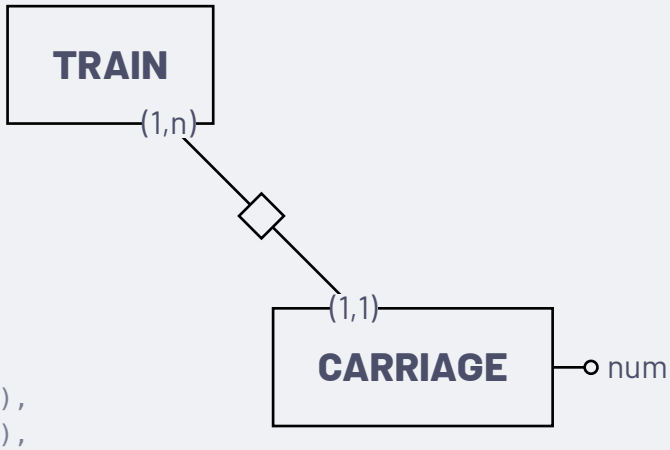


Note carriage and train have been swapped, so that the weak entity has been drawn *before* its corresponding strong entity.

4.2. Unorthodox relation position

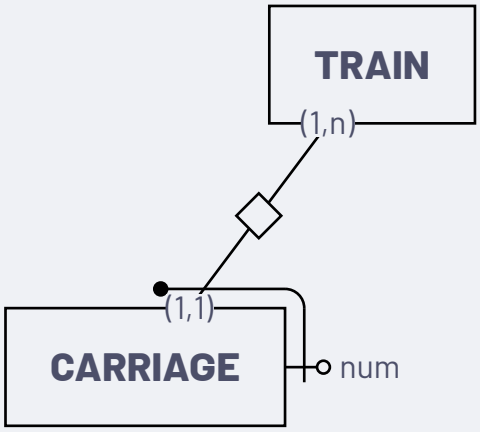
Sometimes, due to the density of entities, the relations cannot be placed right above, left, right or below – the package acknowledges this and automatically handles this situation:

```
#er-diagram({
  entity(
    (0, 0),
    name: "train",
    label: "train",
  )
  entity(
    (4, -4),
    name: "carriage",
    label: "carriage",
    attributes: (east: ("num", )),
  )
  relation(
    entities: ("train", "carriage"),
    cardinality: ("(1,n)", "(1,1)"),
  )
})
```



Weak entity It also automatically handles weak entities in such cases:

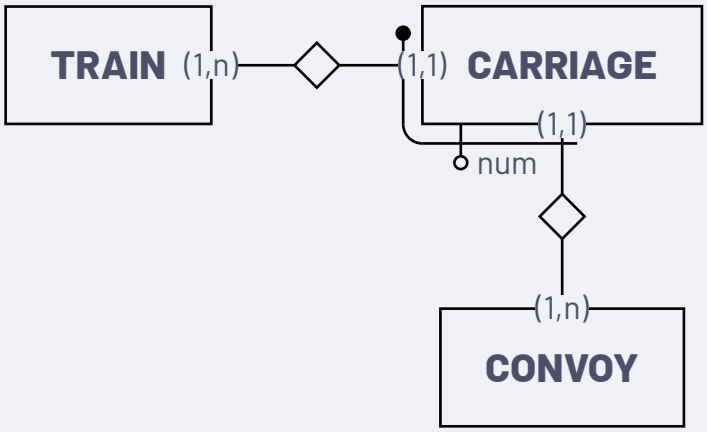
```
#er-diagram({
  entity(
    (3, 0),
    name: "train",
    label: "train",
  )
  entity(
    (0, -4),
    name: "carriage",
    label: "carriage",
    attributes: (east: ("num", )),
    weak-entity: ("num", "train", "CCW")
  )
  relation(
    entities: ("train", "carriage"),
    cardinality: ("(1,n)", "(1,1)"),
  )
})
```



4.3. Exceptions

For some reason, you might need to create a weak entity that involves two entities but can't – or won't – start from the semantically correct attribute; you can still use the cardinal points:

```
#er-diagram({
  entity(
    (6, 0),
    name: "carriage",
    label: "carriage",
    attributes: (south: ("num", )),
    attributes-position: (south: (alignment: left)),
    weak-entity: ("south", "west")
  )
  entity(
    (0, 0),
    name: "train",
    label: "train",
  )
  entity(
    (6,-4),
    name: "convoy",
    label: "convoy",
  )
  relation(
    entities: ("train", "carriage"),
    cardinality: ("(1,n)", "(1,1)"),
  )
  relation(
    entities: ("convoy", "carriage"),
    cardinality: ("(1,n)", "(1,1)"),
  )
})
```



Also do note carriage *is the first entity to be drawn*.

5. User functions

For *all* the generated documentation, refer to `/docs/docs.pdf`.

5.1 dati-basati

Default global settings.

Parameters

```
dati-basati(
  fill: dictionary,
  stroke: dictionary,
  radius: dictionary,
  spacing: dictionary,
  text: dictionary,
  attributes-position: dictionary,
  misc: dictionary,
  body
) -> dictionary
```

fill `dictionary`

The fill of each element.

Default: `(:)`

stroke `dictionary`

The stroke of each element.

Default: `(:)`

radius `dictionary`

The radius of entities, cardinality and hierarchy.

Default: `(:)`

spacing `dictionary`

The spacing of attributes.

Default: `(:)`

text `dictionary`

Text formatting of each element.

Default: `(:)`

attributes-position `dictionary`

The position and direction of attributes.

Default: `(:)`

misc `dictionary`

Miscellanous options.

Default: `(:)`

5.2 er-diagram

Draw an Entity-Relation Diagram. This is the function that wraps all the diagrams.

Parameters

```
er-diagram(
  theme: dictionary,
  ..args: dictionary,
  body: content
) -> content
```

theme `dictionary`

The theme of this diagram.

Default: `(:)`

..args `dictionary`

Arguments directly passed to `#cetz.canvas()`.

body `content`

Body of the diagram.

5.3 entity

Draw an entity.

Parameters

```
entity(
  coordinates: array,
  name: str,
  label: str content,
  attributes: dictionary,
  attributes-position: dictionary,
  primary-key: str array,
  weak-entity: array,
  misc: dictionary,
  ..args: args
) -> content
```

coordinates `array`

Coordinates of the entity.

name `str`

Unique name of the entity.

Default: `""`

label `str` or `content`

Label for the entity (e.g. Person, Book, Employee...).

Default: `"Entity"`

attributes `dictionary`

Attributes of the entity (e.g. id, name, surname, address...).

Default: `(:)`

attributes-position `dictionary`

Fine tune positioning of attributes.

Default: `(:)`

primary-key `str` or `array`

Primary key(s).

Default: `none`

weak-entity `array`

Whether this entity is weak and the corresponding attribute. The format is as follows:

- SQL mirror: `(attribute, strong entity, direction)`
- With cardinal points: `(cardinal pt, cardinal pt, direction)`

They can be mixed.

Default: `none`

misc `dictionary`

Miscellanous options:

- weak-entity-intersection

Default: `none`

..args `args`

Arguments passed to `#std.rect()`.

5.4 subentities

Connect subentities to the superentity.

Parameters

```
subentities(
  hierarchy: "(t,e)" "(p,e)" "(p,o)",
  entity: str,
  subentities: array
) -> content
```

hierarchy `"(t,e)"` or `"(p,e)"` or `"(p,o)"`

The hierarchy of the subentities:

- (t,e) = (total, exclusive)
- (p,e) = (partial, exclusive)
- (p,o) = (partial, overlapping)

Default: `"(t,e)"`

entity `str`

The superentity, the one to which the subentities will connect.

Default: `""`

subentities `array`

The subentities. Can be just one.

Default: `("",)`

5.5 relation

Draw a relation between (sub)entities.

Parameters

```
relation(
  coordinates: array,
  entities: array,
  name: str,
  label: str,
  attributes: dictionary,
  cardinality: array
) -> content
```

coordinates `array`

The specific coordinates of this relation. If left empty, they will the middle point between the two entities.

Default: `none`

entities `array`

Entities that are tied by this relation.

Default: `()`

name `str`

Unique name of relation.

Default: `""`

label `str`

Label for the relation (e.g. “occupies”, “relates”, “is part of”...).

Default: `""`

attributes `dictionary`

Attributes of the relation (e.g. id, name, surname, address...).

Default: `(:)`

cardinality `array`

Cardinality of the given relation:

Default: `()`